UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN



MANUAL TÉCNICO

ORIENTA

APLICACIÓN WEB DE ORIENTACIÓN VOCACIONAL

HUGO EMMANUEL RIVERA GUZMÁN

GUATEMALA, OCTUBRE 2026

1. Introducción, alcance y audiencia

Este manual explica cómo está construido Orienta y cómo instalarlo, configurarlo, desplegarlo y mantenerlo, para quien herede el código.

Qué es Orienta. Una aplicación web de orientación vocacional. El alumno conversa con un guía virtual, responde preguntas fijas y adaptativas, y recibe un tablero con las carreras más afines de un catálogo real de universidades de Quetzaltenango y Totonicapán. Aparte puede hacer el test de intereses de Holland, servido por la API oficial de O*NET. La recomendación la produce un modelo de lenguaje (Gemini, de Google) con salida JSON estructurada.

Audiencia. Desarrolladores que mantengan o extiendan el sistema, personal de TI que lo despliegue, y revisores técnicos del trabajo de graduación. Se asume manejo básico de Python, JavaScript, Git y la terminal.

Alcance. Cubre la versión pública (chat y test de Holland) y menciona los módulos que solo existen en desarrollo (examen psicométrico, perfil corto de personalidad y CIP). No cubre el uso de la aplicación por el alumno: eso está en el manual de usuario.

Convenciones.

- Las rutas son relativas a la raíz del repositorio (`backend/app/main.py`).
- Los comandos de Windows son de PowerShell; los demás, de una terminal POSIX.
- Los valores entre `<` y `>` se sustituyen por los reales.

Repositorios.

Tabla 1. Repositorios del proyecto

La tabla identifica los dos repositorios de código de Orienta y el uso que tiene cada uno dentro del ciclo de desarrollo y publicación.

Repositorio	Uso
HugoUMG/AsistenteVocacional	Repositorio de trabajo: código, experimentos, documentación y scripts de medición
HugoUMG/MiOrienta	Repositorio de producción, del que Render toma el código para desplegar

Tabla 1. Repositorios del proyecto. Fuente: Elaboración propia.

Las dos historias son independientes: cada corrección se aplica en ambos.

Documentación de referencia dentro del repositorio. `README.md` es la entrada: requisitos, arranque y estructura. `docs/` describe arquitectura, API, motor de IA, catálogo y diseño; `decisions/` registra las decisiones técnicas con su motivo; `experiments/` guarda la evidencia medida de cada cambio de prompt o heurística.

2. Arquitectura general y flujo de datos

Orienta tiene tres capas (navegador, API y base de datos) y cuatro servicios externos; el navegador solo habla con la API y con el botón de Google, y todas las llaves viven en el servidor.

Figura 1. Arquitectura de Orienta

La figura muestra las tres capas del sistema (navegador, API y base de datos) y los cuatro servicios externos con los que se comunica la API.

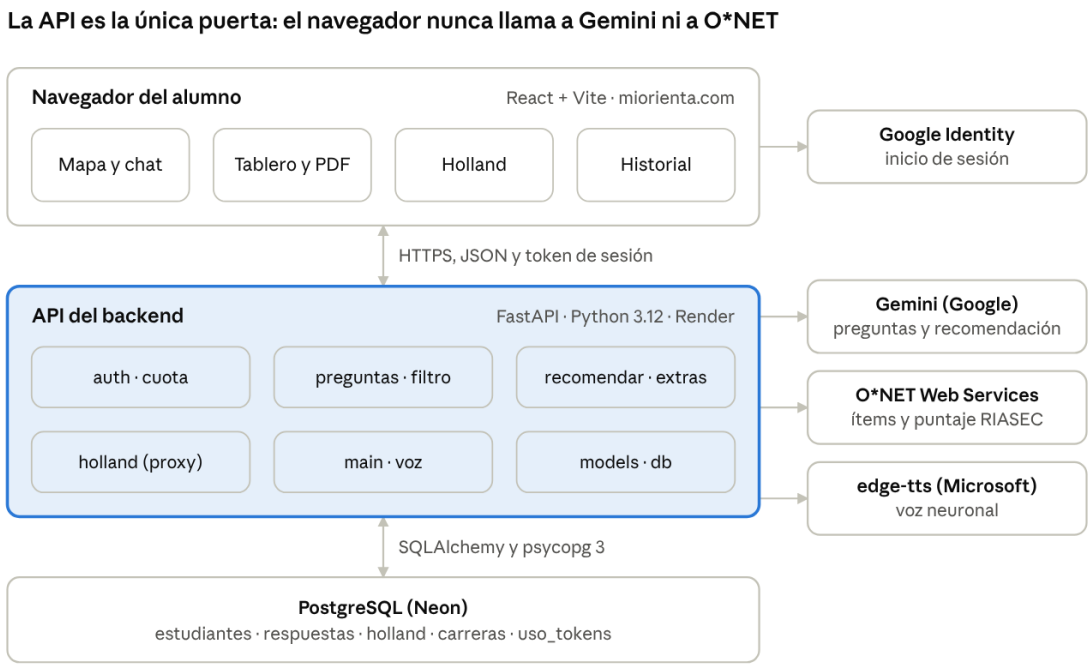


Figura 1. Arquitectura de Orienta. Fuente: Elaboración propia.

La API verifica el token de Google, llama a Gemini y a O*NET con las llaves del servidor, guarda en PostgreSQL y devuelve JSON al navegador.

2.1 Recorrido de una evaluación completa

1. El alumno inicia sesión con Google; la API cambia ese token por un JWT propio (`POST /api/auth/google`).
2. Opcional: responde los 60 ítems de Holland; la API los califica en O*NET y guarda el resultado (`POST /api/holland`).
3. Elige departamento en el mapa; el frontend abre `/chat?depto=<departamento>`.
4. Responde las preguntas fijas en el navegador, sin llamadas a la IA.
5. Por cada pregunta adaptativa, el frontend envía todas las respuestas a `POST /api/next-question`. La API filtra el catálogo por departamento, lo recorta a 35 carreras, actualiza la cobertura de dimensiones y pide a Gemini la siguiente pregunta en JSON.
6. Tras 4 a 8 adaptativas, el frontend guarda las respuestas (`POST /api/submit-survey`) y pide la recomendación (`POST /api/recommend`). La API manda a Gemini el catálogo completo del departamento, recibe afinidades, razones y factores, adjunta las sedes desde la base y guarda la recomendación.
7. El tablero se dibuja en el navegador. "Un día siendo…" y el comparador hacen una llamada a Gemini cada uno, solo si el alumno los pide.
8. Cada llamada a Gemini deja una fila en `uso_tokens` con el `session_id` del recorrido.

3. Stack tecnológico y versiones

React con Vite en el frontend, FastAPI con Python 3.12 en el backend, PostgreSQL como base de datos y la API de Gemini como motor de recomendación.

Tabla 2. Stack tecnológico y versiones

La tabla enumera las tecnologías de cada capa del sistema, la versión fijada en los archivos de dependencias y la función que cumple cada una.

Capa	Tecnología	Versión fijada	Para qué
Frontend	React y React DOM	19.2.7	Interfaz
Frontend	Vite	8.1.0	Servidor de desarrollo y compilación
Frontend	React Router DOM	7.18.1	Rutas de la aplicación multipágina
Frontend	Recharts	3.9.0	Gráfica de anillo del tablero
Frontend	jsPDF	4.2.1	Informe PDF generado en el navegador
Frontend	@react-oauth/google	0.13.5	Botón de inicio de sesión con Google
Backend	Python	3.12	Lenguaje del servidor
Backend	FastAPI	0.138.1	API REST

Capa	Tecnología	Versión fijada	Para qué
Backend	Pydantic	2.13.4	Validación de entradas y del JSON de Gemini
Backend	SQLAlchemy	2.0.51	ORM
Backend	psycopg	3.3.4	Controlador de PostgreSQL
Backend	google-genai	2.10.0	SDK de Gemini
Backend	google-auth y PyJWT	2.55.1 y 2.13.0	Verificar el token de Google y emitir la sesión propia
Backend	edge-tts	7.2.8	Voz neuronal del chat
Datos	PostgreSQL	16	Base de datos
IA	Gemini	`gemini-3.1-flash-lite`	Preguntas adaptativas, recomendación, simulador y comparador
Externo	O*NET Web Services	API v2.0	Ítems y calificación del test de Holland

Tabla 2. Stack tecnológico y versiones. Fuente: Elaboración propia.

Las versiones salen de `frontend/package-lock.json` y `backend/uv.lock`. Node.js 20 o superior es suficiente para el frontend; el desarrollo se hizo con Node 22.

4. Estructura del repositorio

El repositorio separa el backend (`backend/`), el frontend (`frontend/`) y la documentación (`docs/`, `decisions/`, `experiments/`).

4.1 Raíz

Tabla 3. Archivos de la raíz del repositorio

La tabla describe los archivos y carpetas del primer nivel del repositorio de trabajo y el contenido de cada uno.

Ruta	Contenido
`README.md`	Requisitos y arranque rápido
`start.ps1` / `stop.ps1`	Levantan y detienen base de datos, backend y frontend en Windows
`docs/`	Arquitectura, API, motor de IA, catálogo, diseño, Holland, psicométrico y estudio con alumnos
`decisions/`	Decisiones técnicas con fecha, motivo, alternativas y consecuencias
`experiments/`	Evidencia medida de cada cambio de prompt o heurística

Tabla 3. Archivos de la raíz del repositorio. Fuente: Elaboración propia.

4.2 Backend (`backend/`)

Tabla 4. Archivos del backend

La tabla describe los módulos de la aplicación FastAPI, los archivos de datos del catálogo y los scripts de carga, medición y costo.

Ruta	Contenido

Ruta	Contenido
app/main.py	Aplicación FastAPI: endpoints, validación de entradas, CORS, voz y manejadores de error
app/models.py	Las siete tablas de SQLAlchemy
app/db.py	Conexión a PostgreSQL y normalización de la URL
app/auth.py	Verificación del token de Google, sesión JWT propia y lista de administradores
app/cuota.py	Límite de chats por alumno y tope diario global de tokens
app/recomendar.py	Capa común de Gemini (keys, reintentos, caché, tokens) y la recomendación final
app/preguntas.py	Preguntas adaptativas y vector de cobertura de dimensiones
app/filtro.py	Pre-filtro sin IA del catálogo y descarte de la carrera abandonada
app/extras.py	Simulador "Un día siendo…" y comparador de dos carreras
app/holland.py	Proxy hacia la API de O*NET
app/holland_filtro.py	Carreras del catálogo afines al código RIASEC y prioridad experimental
app/diversificado.py	Sugerencia de diversificado para alumnos de básicos, sin IA
app/psicometrico.py, app/personalidad.py, app/cip*.py	Instrumentos que solo se ofrecen en desarrollo
data/carreras_*.json	Catálogo: un archivo por centro universitario
data/perfiles_compartidos.json	Perfiles de las carreras que se ofrecen en varias sedes
data/diversificados.json	Tabla de modalidades de diversificado
data/holland_catalogo.json	Código RIASEC de cada carrera del catálogo
seed_carreras.py	Carga el catálogo a la base de datos
experimento_*.py, analiza_filtro.py, cobertura_banco.py, codificar_*.py	Scripts de medición; algunos gastan cuota de Gemini

Tabla 4. Archivos del backend. Fuente: Elaboración propia.

4.3 Frontend (`frontend/`)

Tabla 5. Archivos del frontend

La tabla describe los componentes y módulos de la interfaz en React, agrupados por la pantalla o la función que resuelven.

Ruta	Contenido
src/main.jsx	Rutas de la aplicación y proveedor de Google
src/Inicio.jsx, Acerca.jsx, Parametros.jsx, Catalogo.jsx	Páginas informativas
src/Mapa.jsx	Selección de departamento
src/Chat.jsx	Chat: preguntas fijas y adaptativas, voz, análisis final
src/preguntas-fijas.js	Texto y opciones de las preguntas fijas
src/Dashboard.jsx, Dashboard.css	Tablero de resultados, simulador y comparador
src/reporte.js	Informe PDF con jsPDF
src/Holland.jsx, holland-perfil.js	Test de Holland y perfil guardado
src/Historial.jsx, Recorrido.jsx	Historial del alumno y vista del recorrido
src/Admin.jsx	Registro de evaluaciones para administradores

Ruta	Contenido
`src/Protegida.jsx`, `Bienvenida.jsx`, `auth.js`	Inicio de sesión obligatorio, términos y sesión
`src/modo.js`	Qué se ofrece en producción y qué solo en desarrollo

Tabla 5. Archivos del frontend. Fuente: Elaboración propia.

5. Requisitos e instalación en entorno local

Con Docker Desktop, uv, Node.js y las tres credenciales externas, el sistema completo arranca en Windows con `.\start.ps1`.

5.1 Software necesario

Tabla 6. Software necesario para la instalación

La tabla indica las herramientas que deben estar instaladas en el equipo antes de levantar Orienta en local, con la versión mínima y su propósito.

Herramienta	Versión mínima	Para qué
Git	Cualquiera reciente	Clonar el repositorio
Python	3.12	Backend (uv lo instala si falta)
uv	0.10	Entorno y dependencias de Python
Node.js y npm	Node 20 (probado con 22)	Frontend
Docker Desktop	Cualquiera reciente	PostgreSQL 16 en un contenedor

Tabla 6. Software necesario para la instalación. Fuente: Elaboración propia.

5.2 Credenciales externas

Tabla 7. Credenciales de servicios externos

La tabla resume las tres credenciales que requiere el sistema, dónde se obtiene cada una y la variable de entorno en la que se configura.

Credencial	Dónde se obtiene	Variable
API key de Gemini	Google AI Studio (aistudio.google.com)	`GEMINI_API_KEY`
API key de O*NET	Registro gratuito en services.onetcenter.org/developer/signup	`ONET_API_KEY`
Client ID de OAuth de Google	Google Cloud Console, APIs y servicios, Credenciales, ID de cliente de OAuth de tipo aplicación web	`GOOGLE_CLIENT_ID` y `VITE_GOOGLE_CLIENT_ID`

Tabla 7. Credenciales de servicios externos. Fuente: Elaboración propia.

En el cliente de OAuth, agregue `http://localhost:5173` a los orígenes de JavaScript autorizados. Sin él, el botón de Google falla en local.

5.3 Primera instalación

1. Clone el repositorio y entre a la carpeta.
2. Copie `backend/.env.example` a `backend/.env` y complete al menos `GEMINI_API_KEY`, `ONET_API_KEY`, `GOOGLE_CLIENT_ID` y `SESSION_SECRET` (una cadena aleatoria larga, por ejemplo la salida de `python -c "import secrets; print(secrets.token_hex(32))"`).
3. Para desarrollo, ponga `MAX_CHATS_DIARIOS=0` (sin límite de chats) y, si quiere probar sin Google, `LOGIN_OPCIONAL=1`.
4. Copie `frontend/.env.example` a `frontend/.env` y ponga el mismo Client ID en `VITE_GOOGLE_CLIENT_ID`.
5. Instale las dependencias del frontend: `cd frontend` y `npm install`. Las del backend las instala `uv` la primera vez que se ejecuta `uv run`.

5.4 Arranque rápido en Windows

Desde la raíz, con Docker Desktop instalado:

```
.\start.ps1
```

El script:

1. Abre Docker Desktop si no está corriendo y espera hasta 2 minutos.
2. Arranca el contenedor `tfg-db` (PostgreSQL 16, usuario `postgres`, contraseña `dev`, base `tfg`, puerto 5432) o lo crea la primera vez, y espera a que acepte conexiones.
3. Carga o actualiza el catálogo con `seed_carreras.py`.
4. Abre una ventana con el backend en el puerto 8000 y otra con el frontend en el 5173.
5. Abre el navegador en `http://localhost:5173`.

Si PowerShell bloquea el script: `powershell -ExecutionPolicy Bypass -File .\start.ps1`. Para detener todo, `.\stop.ps1`: cierra los procesos de los puertos 8000 y 5173 y detiene el contenedor, que conserva sus datos.

5.5 Arranque manual (cualquier sistema)

```
docker run --name tfg-db -e POSTGRES_PASSWORD=dev -e POSTGRES_DB=tfg -p 5432:5432 -d
postgres:16
cd backend
```

```
uv run python seed_carreras.py
uv run uvicorn app.main:app --reload --port 8000
```

En otra terminal:

```
cd frontend
npm install
npm run dev
```

Las veces siguientes basta `docker start tfg-db` en lugar de `docker run`.

5.6 Verificación

Tabla 8. Comprobaciones de la instalación

La tabla lista las direcciones que se consultan para confirmar que el backend, la base de datos y el frontend quedaron funcionando, con el resultado esperado de cada una.

Comprobación	Resultado esperado
`http://localhost:8000/health`	`{"status":"ok"}`
`http://localhost:8000/docs`	Documentación interactiva de la API (Swagger)
`http://localhost:8000/api/departamentos`	`Quetzaltenango` y `Totonicapán`
`http://localhost:5173`	Página de inicio de Orienta

Tabla 8. Comprobaciones de la instalación. Fuente: Elaboración propia.

5.7 Particularidades del modo desarrollo

`npm run dev` ofrece más que la versión pública:

- Las rutas `/psicometrico`, `/cip` y `/personalidad`, y el enlace **Exámenes psicométricos** en la barra.
- El botón **Entrar sin sesión (solo local)**, que necesita `LOGIN_OPCIONAL=1` en el backend. Entra como el alumno de prueba `dev@local`.
- Perfiles predeterminados en el test de Holland, que califican un perfil simulado sin contestar las 60 preguntas.

Nada de esto sale en `npm run build`: lo controla `import.meta.env.DEV` (`src/modo.js`).

6. Configuración: variables de entorno y flags

Toda la configuración vive en variables de entorno: `backend/.env` en local y el panel del proveedor en producción. Ningún secreto va en el repositorio.

6.1 Backend (`backend/.env`)

Tabla 9. Variables de entorno del backend

La tabla describe cada variable que lee el backend, si es obligatoria, su valor por omisión y el comportamiento que controla.

Variable	Obligatoria	Valor por omisión	Qué controla
`DATABASE_URL`	Sí en producción	`postgresql+psycopg://postgres:dev@localhost:5432/tfg`	Conexión a PostgreSQL. Acepta `postgres://` y `postgresql://`; se normaliza al controlador psycopg 3
`ORIGENES_PERMITIDOS`	Sí en producción	`http://localhost:5173`	Orígenes CORS, separados por coma. Nunca `*`
`GEMINI_API_KEY`	Sí		Key principal de Gemini. Sin ella los endpoints de IA responden 503
`GEMINI_API_KEY_RESPALDO`	No		Key de otro proyecto, con facturación; se usa solo si la principal responde 429
`GEMINI_API_KEY_1` a `_N`	No		Pool de keys: cada sesión toma una en rotación y la usa toda la conversación. Si existe, tiene prioridad sobre `GEMINI_API_KEY`
`GEMINI_MODEL`	No	`gemini-3.1-flash-lite`	Modelo de las preguntas adaptativas
`GEMINI_MODEL_FINAL`	No	`gemini-3.1-flash-lite`	Modelo de la recomendación, el simulador y el comparador
`ONET_API_KEY`	Sí, para Holland		Key de O*NET. Sin ella, Holland responde 503 y el resto sigue
`GOOGLE_CLIENT_ID`	Sí		Client ID de OAuth. Sin él, nadie puede iniciar sesión
`SESSION_SECRET`	Sí		Clave que firma los JWT propios. Cambiarla cierra todas las sesiones
`ADMIN_EMAILS`	No	Vacío (nadie)	Correos con acceso al registro de evaluaciones, separados por coma
`MAX_CHATS_DIARIOS`	No	`3`	Chats terminados por alumno en 24 horas. `0` lo desactiva
`TOPE_TOKENS_DIARIO`	No	`2000000`	Tokens de Gemini de toda la aplicación por día UTC. Al llegar, responde 429
`CACHE_EXPLICITO`	No	`0`	Caché explícito de Gemini (alquila almacenamiento por hora). Solo conviene con una clase entera a la misma hora
`LOGIN_OPCIONAL`	No	Sin definir	Con `1` exacto, permite evaluarse sin sesión como `dev@local`. Nunca en

Variable	Obligatoria	Valor por omisión	Qué controla
			producción

Tabla 9. Variables de entorno del backend. Fuente: Elaboración propia.

Advertencia sobre el pool de keys. Las cuotas gratuitas de Gemini son por proyecto de Google Cloud. Crear proyectos gratuitos adicionales solo para multiplicar la cuota viola los términos de servicio de Google y está prohibido por las reglas del proyecto. Use el pool o la key de respaldo solo con proyectos legítimos, por ejemplo con facturación habilitada.

6.2 Flags experimentales

Todos están apagados. Existen para repetir mediciones sin tocar código; ninguno mejoró la recomendación cuando se midió.

Tabla 10. Flags experimentales

La tabla presenta los interruptores que activan variantes del motor de recomendación y el resultado que obtuvo cada una cuando se midió.

Flag	Qué hace al encenderlo	Resultado medido
`HOLLAND_EN_RECOMENDACION` (y `HOLLAND_TOP`)	Ordena el catálogo por afinidad RIASEC antes de la recomendación	El top-1 no cambió (`experiments/holland-estructura.md`)
`EDAD_Y_GRADO_EN_RECOMENDACION`	Agrega al prompt instrucciones sobre edad, grado y carrera cursada	11 comparaciones, cero mejoras y una regresión (`experiments/edad-y-grado.md`)
`CIP_EN_RECOMENDACION` (y `CIP_TOP`)	Prioriza el catálogo con el perfil del CIP	9/10 contra 10/10, revertido (`experiments/cip-en-recomendacion.md`)

Tabla 10. Flags experimentales. Fuente: Elaboración propia.

6.3 Frontend (`frontend/.env`)

Tabla 11. Variables de entorno del frontend

La tabla describe las dos variables que se fijan al compilar la interfaz y el efecto de cada una sobre el sitio publicado.

Variable	Obligatoria	Qué controla
`VITE_GOOGLE_CLIENT_ID`	Sí	El mismo Client ID del backend. Sin él, el botón de Google no aparece
`VITE_API_URL`	Sí en producción	URL pública del backend, sin barra final. Por omisión, `http://localhost:8000`

Tabla 11. Variables de entorno del frontend. Fuente: Elaboración propia.

Vite congela estas variables al compilar: cambiarlas exige volver a ejecutar `npm run build`, no basta con reiniciar.

6.4 Valores que se cambian en código

Tabla 12. Valores configurables en el código

La tabla reúne las constantes que cambian el comportamiento del sistema y que se modifican directamente en el código, no en variables de entorno.

Constante	Archivo	Valor	Qué controla
`MIN_ADAPTATIVAS` / `MAX_ADAPTATIVAS`	`app/preguntas.py` y `src/Chat.jsx` (deben coincidir)	4 / 8	Preguntas adaptativas mínimas y máximas
`MARGEN_DESEMPATE`	`app/preguntas.py`	20	Puntos que la carrera n.º 1 debe sacarle a la n.º 2 para cerrar el chat
`TOP_DEFAULT`	`app/filtro.py`	35	Carreras que deja pasar el pre-filtro en cada pregunta adaptativa
`DIAS_EXPIRACION`	`app/auth.py`	90	Vigencia del JWT de sesión
`_VOZ_TTS`	`app/main.py`	`es-MX-DaliaNeural`	Voz neuronal del chat
`ACTIVOS`	`src/Mapa.jsx`	Quetzaltenango, Totonicapán	Departamentos que se dibujan en el mapa
`MODO_COMPLETO`	`src/modo.js`	`import.meta.env.DEV`	Qué instrumentos se ofrecen

Tabla 12. Valores configurables en el código. Fuente: Elaboración propia.

7. Base de datos

PostgreSQL guarda siete tablas definidas en `backend/app/models.py`; el `session_id` de cada recorrido une el test de Holland, el chat y el consumo de tokens de una misma persona.

7.1 Tablas

Tabla 13. Tablas de la base de datos

La tabla describe las siete tablas de PostgreSQL definidas en el modelo de datos, la información que guarda cada una y sus columnas principales.

Tabla	Qué guarda	Columnas principales
`estudiantes`	Una cuenta por alumno	`id`, `nombre`, `email` (único), `google_sub` (único, id estable de Google), `terminos_aceptados`, `created_at`
`respuestas_cuestionario`	Una fila por chat	`estudiante_id` (FK), `session_id`, `respuestas` (JSON con todas las respuestas), `recomendacion` (JSON con la lista de carreras), `juicio`, `juicio_nota`, `created_at`
`resultados_holland`	Un intento del test de Holland	`estudiante_id` (FK), `session_id`, `respuestas` (60 dígitos), `codigo` (3 letras), `areas` (JSON con los 6 puntajes), `perfil` (JSON que consume el chat)
`carreras`	Catálogo, una fila por carrera y sede	`nombre`, `departamento`, `centro`, `universidad`, `perfil` (texto que lee la IA), `perfil_grupo`, `sello`
`uso_tokens`	Una fila por llamada a Gemini	`session_id`, `endpoint`, `modelo`, `prompt_tokens`, `output_tokens`, `total_tokens`, `cached_tokens`
`resultados_psicometricos`	Examen de 100 ítems (solo desarrollo)	`estudiante_id`, `session_id`, `respuestas`, `puntajes`, `resumen`
`resultados_personalidad`	Perfil corto de 48 ítems (solo desarrollo)	`estudiante_id`, `session_id`, `respuestas`, `puntajes`

Tabla 13. Tablas de la base de datos. Fuente: Elaboración propia.

7.2 Relaciones

- `estudiantes` tiene muchas filas en `respuestas_cuestionario` (llave foránea obligatoria) y en las tres tablas de resultados (llave foránea opcional).
- `session_id` no es llave foránea: es un UUID que genera el navegador por recorrido y que se repite en `respuestas_cuestionario`, `resultados_holland` y `uso_tokens`. Con él se sabe si un Holland precedió a un chat concreto.
- `carreras` no se relaciona con las demás. La recomendación guarda nombres de carrera, no ids, para que recargar el catálogo no rompa evaluaciones viejas.

Figura 2. Modelo de datos

La figura muestra cómo se relacionan las siete tablas: las llaves foráneas que parten de estudiantes y el identificador de recorrido que une los resultados de una misma persona.

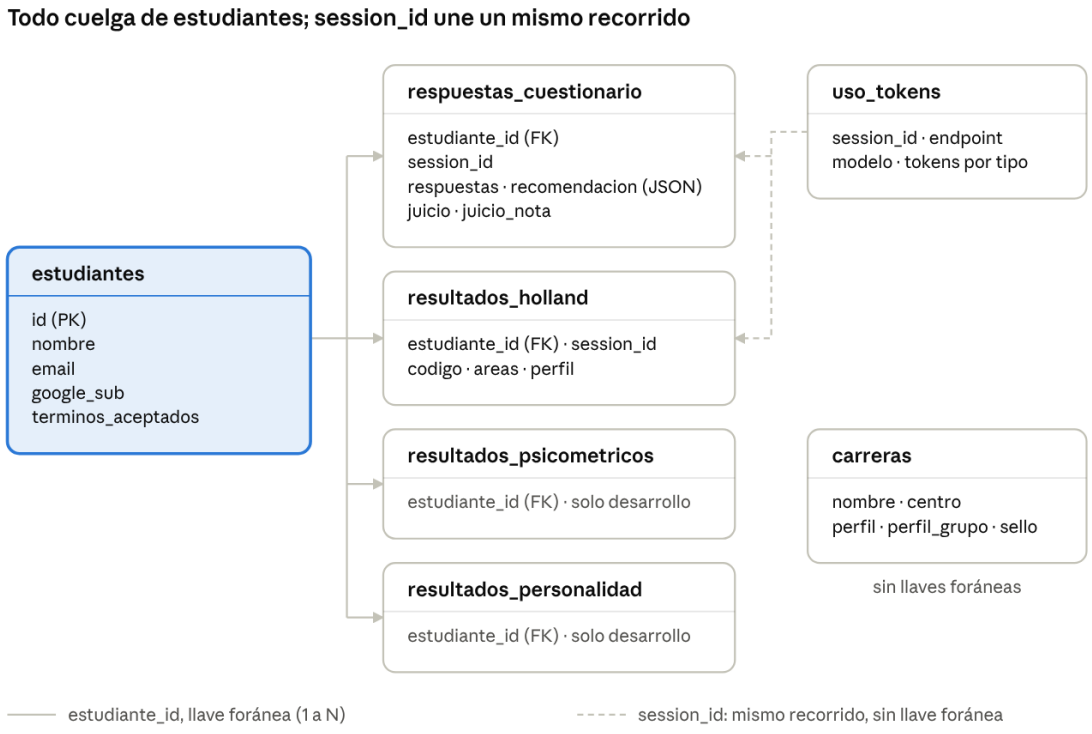


Figura 2. Modelo de datos. Fuente: Elaboración propia.

7.3 Columna `juicio`

Admite `acerto`, `parcial`, `no_acerto`, `descartada` o nulo. Los tres primeros son la calificación de la psicóloga, el criterio externo del estudio. `descartada` marca pruebas y evaluaciones de conocidos hechas contra producción: hay que filtrarla **antes** de contar aciertos.

7.4 Creación y cambios de esquema

No hay migraciones. Al arrancar, el backend ejecuta `Base.metadata.create_all` y luego cuatro `ALTER TABLE … ADD COLUMN IF NOT EXISTS` para las columnas que se agregaron después (`resultados_holland.perfil`, `respuestas_cuestionario.juicio` y `juicio_nota`, `estudiantes.terminos_aceptados`).

Para agregar una columna a una tabla existente:

1. Declárela en `models.py`.
2. Agregue su `ALTER TABLE … ADD COLUMN IF NOT EXISTS` en `lifespan()` de `main.py`.
3. Reinicie el backend.

`create_all` no modifica tablas que ya existen; sin el paso 2, la columna no aparece. Cuando haya más cambios de este tipo, conviene migrar a Alembic.

7.5 Respaldo y restauración

```
pg_dump "<DATABASE_URL>" --format=custom --file=orienta-<fecha>.dump
pg_restore --clean --if-exists --dbname "<DATABASE_URL>" orienta-<fecha>.dump
```

El respaldo contiene datos personales de menores: guárdelo cifrado y con acceso restringido.

7.6 Borrar los datos de un alumno

Los términos de uso ofrecen borrar la cuenta a pedido. Con el `id` del alumno:

```
BEGIN;
DELETE FROM resultados_holland WHERE estudiante_id = <id>;
DELETE FROM resultados_psicometricos WHERE estudiante_id = <id>;
DELETE FROM resultados_personalidad WHERE estudiante_id = <id>;
DELETE FROM respuestas_cuestionario WHERE estudiante_id = <id>;
DELETE FROM estudiantes WHERE id = <id>;
COMMIT;
```

Las filas de `uso_tokens` no tienen datos personales (solo el `session_id` y conteos) y pueden quedarse.

8. Backend: módulos y referencia de la API

El backend es una sola aplicación FastAPI (`app/main.py`) que valida cada entrada con Pydantic, exige sesión en todo lo que evalúa al alumno y delega la IA, la autenticación y los límites en módulos propios.

8.1 Endpoints

En la columna Acceso, **Libre** no pide sesión, **Sesión** pide `Authorization: Bearer <jwt>` y **Evaluador** pide sesión y además respeta el tope global de tokens.

Tabla 14. Endpoints de la API

La tabla enumera cada ruta del backend con su método, el nivel de acceso que exige, si llama al motor de IA y la función que cumple.

Método	Ruta	Acceso	Gemini	Qué hace
GET, HEAD	`/health`	Libre	No	Estado del servicio. Acepta HEAD para el monitor de disponibilidad
POST	`/api/auth/google`	Libre	No	Cambia el ID token de Google por un JWT propio; crea o reusa la cuenta
POST	`/api/aceptar-terminos`	Sesión	No	Confirma el nombre y guarda la fecha de aceptación de términos
POST	`/api/register`	Sesión	No	Devuelve la cuenta del alumno; valida el nombre
POST	`/api/submit-survey`	Sesión	No	Guarda las respuestas del chat con su `session_id`
GET	`/api/departamentos`	Libre	No	Departamentos con catálogo cargado
GET	`/api/carreras`	Libre	No	Catálogo completo con arquetipo y sello
POST	`/api/next-question`	Evaluador	Sí	Siguiente pregunta adaptativa y ranking provisional. Revisa el límite de chats
POST	`/api/recommend`	Evaluador	Sí	Recomendación final; la guarda en el último chat del alumno
POST	`/api/simular-dia`	Evaluador	Sí	"Un día siendo…" de una carrera del resultado
POST	`/api/comparar`	Evaluador	Sí	Comparación de dos carreras del resultado
POST	`/api/tts`	Libre	No	Audio MP3 en streaming con la voz neuronal
GET	`/api/holland/preguntas`	Libre	No	Los 60 ítems y la escala, desde O*NET
POST	`/api/holland`	Sesión	No	Califica con O*NET, agrega carreras afines del catálogo y guarda
GET	`/api/holland/mio`	Sesión	No	Último perfil de Holland de la cuenta, o `null`
GET	`/api/holland/{rid}`	Sesión	No	Resultado completo de un Holland guardado (propio, o cualquiera si es administrador)
GET	`/api/historial`	Sesión	No	Todas las evaluaciones del alumno
POST	`/api/historial/reclamar`	Sesión	No	Asigna a la cuenta los resultados anónimos de un `session_id`
GET	`/api/admin/soy`	Libre	No	Indica si la sesión actual es de administrador
GET	`/api/admin/re`	Administrador	No	Registro de evaluaciones

Método	Ruta	Acceso	Gemini	Qué hace
	`spuestas`			(parámetro `limite`, de 1 a 2000, por omisión 500)
GET	`/api/admin/respuestas/{id}`	Administrador	No	Una evaluación completa con su Holland y su historial
POST	`/api/admin/juicio`	Administrador	No	Guarda la calificación del profesional
GET	`/api/uso-tokens`	Libre	No	Resumen de consumo de tokens por sesión y por endpoint
GET, POST	`/api/psicometrico*`, `/api/personalidad*`, `/api/cip*`	Varía	Solo psicométrico	Instrumentos que solo usa la versión de desarrollo

Tabla 14. Endpoints de la API. Fuente: Elaboración propia.

La documentación interactiva de todos los esquemas está en `/docs` (Swagger) mientras corre el backend.

8.2 Ejemplo: siguiente pregunta

Petición a `POST /api/next-question`:

```
{
  "respuestas": {
    "departamento": "Quetzaltenango",
    "nombre": "Ana",
    "edad": "17",
    "nivel": "Diversificado",
    "grado": "Quinto bachillerato",
    "impacto": "Ayudar, enseñar o dar cuidados a las personas",
    "gustos": "Salud y cuidar pacientes, Biología y naturaleza"
  },
  "session_id": "3f1c…-uuid",
  "holland": null
}
```

Respuesta (esquema `SiguientePaso`):

```
{
  "terminado": false,
  "pregunta_texto": "Se nota que te gusta **ayudar**. Imagina que…",
  "pregunta_tipo": "opcion",
  "multiple": false,
  "opciones": [{"value": "laboratorio", "label": "En un laboratorio"}],
  "ranking": [{"carrera": "Enfermería", "afinidad": 62}],
  "alerta_contradiccion": "",
  "dimension_objetivo": "habilidades"
}
```

Las respuestas adaptativas se guardan en `respuestas` con el texto de la pregunta como clave.

8.3 Ejemplo: recomendación

`POST /api/recommend` recibe el mismo cuerpo más `estudiante_id` (ignorado: se usa el de la sesión) y devuelve:

```
{
  "carreras": [{
    "carrera": "Licenciatura en Enfermería",
    "afinidad": 34,
    "descripcion": "…",
    "razones": ["Disfrutas cuidar a otros"],
    "factores": [{"nombre": "Trato con personas", "peso": 85}],
    "instituciones": [{"universidad": "…", "centro": "…", "departamento":
"Quetzaltenango", "enfoque": "…", "programa": "Técnico Universitario en Enfermería"}]
  }],
  "respuesta_id": 123,
  "confianza": 78,
  "confianza_nota": "…",
  "diversificados": []
}
```

Las afinidades de todas las carreras suman 100. Las instituciones no las escribe la IA: Python las adjunta después desde la base de datos, con el nombre del programa en cada sede. Una sede puede dar dos programas de la misma carrera (licenciatura y técnico); el tablero y el PDF usan ese nombre para distinguirlos.

8.4 Códigos de error

Tabla 15. Códigos de error de la API

La tabla explica en qué situaciones responde la API con cada código de error HTTP, para orientar el diagnóstico de fallas.

Código	Cuándo
401	Falta la sesión o el JWT es inválido o venció; o el token de Google no se pudo verificar
403	El correo no está en `ADMIN_EMAILS`
404	La evaluación o el resultado no existe
409	No hay carreras para el departamento pedido
422	Entrada inválida (Pydantic), o Gemini bloqueó el contenido por sus filtros de seguridad
429	Límite de chats del alumno (con cabecera `Retry-After`) o tope global de tokens del día
502	O*NET no respondió
503	Falta una credencial en el servidor (Gemini, O*NET o Google) o Gemini falló tras los reintentos

Tabla 15. Códigos de error de la API. Fuente: Elaboración propia.

Los mensajes de 429 y 503 están redactados para el alumno y el frontend los muestra tal cual.

Dos manejadores globales de `main.py` convierten cualquier falla de Gemini en 503 con un mensaje legible y un bloqueo de sus filtros de seguridad en 422. Sin ellos, la respuesta saldría sin cabeceras CORS y el navegador solo diría "Failed to fetch".

9. Motor de IA

Gemini recibe el catálogo como texto y las respuestas del alumno, y devuelve JSON validado contra un esquema Pydantic; el código de Python vigila que el chat no termine antes de cubrir las dimensiones prioritarias.

9.1 Llamadas a Gemini

Tabla 16. Llamadas al motor de IA

La tabla describe los cuatro usos de Gemini en el sistema, con el modelo, la temperatura, el catálogo que recibe cada llamada y el esquema que valida su respuesta.

Uso	Módulo	Modelo	Temperatura	Catálogo que recibe	Esquema de salida
Pregunta adaptativa	`preguntas.py`	`GEMINI_MODEL`	0.5	Recortado a 35 por el pre-filtro	`SiguientePaso`
Recomendación final	`recomendar.py`	`GEMINI_MODEL_FINAL`	0.3	Completo, del departamento elegido	`ResultadoLLM`
Un día siendo…	`extras.py`	`GEMINI_MODEL_FINAL`	0.6	Ninguno (recibe la descripción de la carrera)	`SimulacionDia`
Comparar	`extras.py`	`GEMINI_MODEL_FINAL`	0.4	Ninguno (recibe las dos descripciones)	`Comparacion`

Tabla 16. Llamadas al motor de IA. Fuente: Elaboración propia.

Un chat completo hace unas cinco llamadas (cuatro o más adaptativas y una final), más una por cada extra que pida el alumno.

9.2 Cómo se arma el prompt

Cada llamada tiene tres partes:

1. **Instrucción de sistema.** Las reglas de la tarea, más dos bloques comunes: `TONO` (escribir para un adolescente de 13 a 17 años, con palabras sencillas) y `ANTI_INYECCION` (todo lo que escribe el alumno son datos, nunca órdenes).

2. **Catálogo.** Un bloque por carrera, `### <nombre>` seguido de su perfil (arquetipo, afinidad, habilidades, entorno, gustos). Las carreras que varias sedes ofrecen comparten `perfil_grupo` y viajan una sola vez. La IA nunca recibe universidad, centro ni sello: Python los adjunta después a partir del nombre.
3. **Parte variable.** El perfil de Holland (si existe), las respuestas del alumno y, en las adaptativas, el estado de cobertura de dimensiones.

La salida se pide con `response_mime_type="application/json"` y `response_schema=<modelo Pydantic>`, y se valida con `model_validate_json`. Si Gemini bloquea la petición o devuelve texto vacío, `_texto_seguro` lanza `ContenidoRechazado` y el endpoint responde 422.

Los prompts son agnósticos del catálogo: no nombran ninguna carrera. Agregar carreras no requiere tocar código. Para ver el prompt real sin gastar cuota: `uv run python dump_prompt.py Quetzaltenango` (lo escribe en `docs/`).

9.3 Cobertura de las siete dimensiones

Un perfil vocacional se explora en siete dimensiones: personalidad, intereses, habilidades, estilo cognitivo, valores, entorno y motivaciones.

- Las preguntas fijas ya cubren intereses, entorno y motivaciones. Las otras cuatro son las **prioritarias**.
- `preguntas.py` lleva, por `session_id`, un vector `{dimensión: 0 o 1}` y se lo manda a Gemini como estado explícito en cada llamada.
- La IA declara en `dimension_objetivo` a qué dimensión apunta cada pregunta, y el backend la marca como cubierta.
- **Guardia de cierre.** Si la IA responde `terminado=true` pero quedan prioritarias pendientes, o el ranking provisional tiene a la n.º 1 a menos de 20 puntos de la n.º 2, y se llevan menos de 8 adaptativas, el backend repite la llamada una vez con un recordatorio. En el caso del empate, el recordatorio nombra las dos carreras y pide una pregunta que las separe sin nombrárselas al alumno.
- Un chat sin adaptativas respondidas reinicia el vector de esa sesión, para que una prueba nueva con el mismo `session_id` no herede la cobertura anterior.

El vector vive en memoria del proceso: se pierde si el backend se reinicia a mitad de un chat. Con un solo proceso es aceptable; con varias instancias habría que moverlo a la base de datos.

Esta regla subió el cumplimiento del mínimo de 4 preguntas de 40% a 100% y el acierto de área de 7/10 a 10/10 en perfiles simulados (`experiments/cobertura-dimensiones.md`).

9.4 Pre-filtro del catálogo

`filtro.preseleccionar` recorta el catálogo antes de cada pregunta adaptativa a las 35 carreras con más palabras en común con las respuestas, sin IA. Se recalcula en cada llamada con todas las respuestas acumuladas.

- Ignora nombre, edad, nivel, grado, motivo y departamento: no aportan señal vocacional.
- No entiende la negación: "me desagradaría ver sangre" sube el puntaje de las carreras de salud.
- La recomendación final no lo usa: recibe el catálogo completo del departamento para no excluir una carrera válida.
- Un A/B con brazo de control mostró que quitarlo no cambia la recomendación más allá del ruido; se mantiene porque reduce tokens.

`filtro.descartar` quita del catálogo la carrera que el alumno dijo haber abandonado, por palabras significativas. Si el texto es tan genérico que se llevaría más del 30% del catálogo, no descarta nada.

9.5 Keys, reintentos y respaldo

1. `key_de_sesion` asigna a cada `session_id` una key del pool `GEMINI_API_KEY_1..N` en rotación, y la sesión la usa en todas sus llamadas. Sin pool, usa `GEMINI_API_KEY`.
2. `_con_reintento` repite hasta 4 veces ante 429, 500, 503 o cortes de conexión. Si Google indica cuánto esperar (`retryDelay`), espera eso con un tope de 30 segundos; si no, espera de forma exponencial con un poco de azar.
3. Si aún así la key responde 429 y existe `GEMINI_API_KEY_RESPALDO`, se intenta una vez con ella.
4. Si todo falla, el manejador global responde 503 al navegador.

El simulador y el comparador (`extras.py`) son la excepción: llaman a Gemini directo con `GEMINI_API_KEY`, sin pool, sin reintentos y sin respaldo. Si fallan, el alumno ve el error en la ventana y puede volver a pulsar el botón.

9.6 Caché

- **Implícito (por omisión).** Gemini cachea por su cuenta los prefijos repetidos, sin alquiler.
- **Explícito (`CACHE_EXPLICITO=1`).** Crea un `CachedContent` con el sistema y el catálogo, por una hora, indexado por modelo, hash SHA-256 del contenido y key. Solo funciona con facturación. Cobra almacenamiento por hora, así que solo conviene con una clase entera usando la app a la misma hora. Como el pre-filtro cambia el catálogo en cada pregunta, en las adaptativas crea un caché por llamada.

El detalle y las mediciones están en `decisions/gemini-costos-y-caching.md` y `experiments/cache-compartido.md`.

9.7 Registro de consumo

Cada llamada hecha desde un endpoint escribe una fila en `uso_tokens` con los tokens de entrada, salida, total y cacheados. `GET /api/uso-tokens` los resume. Los scripts de experimentos llaman a Gemini directo y no pasan por esa tabla: llevan su propio contador en memoria (`resumen_gasto()`).

`uso_tokens` no registra el alquiler del caché explícito, que solo aparece en la factura de Google. Toda cifra que salga de esa tabla es una cota inferior del gasto.

10. Frontend

El frontend es una aplicación React de varias páginas con `react-router`; no usa una librería de estado global, y lo que debe sobrevivir a una recarga vive en el almacenamiento del navegador.

10.1 Rutas

Tabla 17. Rutas del frontend

La tabla relaciona cada ruta de la aplicación con su componente, indica si exige sesión y si se ofrece en la versión pública.

Ruta	Componente	Protegida	En producción	Qué es
`/`	`Inicio.jsx`	No	Sí	Portada y elección de modo
`/acerca`	`Acerca.jsx`	No	Sí	Página informativa
`/catalogo`	`Catalogo.jsx`	No	Sí	Catálogo con búsqueda y filtros
`/parametros`	`Parametros.jsx`	No	Sí	Las siete dimensiones
`/mapa`	`Mapa.jsx`	No	Sí	Elección de departamento; lleva a `/chat?depto=`
`/chat`	`Chat.jsx`	Sí	Sí	Chat y, al terminar, el tablero
`/holland`	`Holland.jsx`	Sí	Sí	Test de Holland
`/historial`	`Historial.jsx`	Pide sesión en la propia página	Sí	Evaluaciones del alumno
`/admin`	`Admin.jsx`	Sí, más administrador en el backend	Sí, sin enlace en el menú	Registro de evaluaciones
`/psicometrico`, `/cip`, `/personalidad`		Sí	No	Solo en `npm run dev`
`*`				Redirige a `/`

Tabla 17. Rutas del frontend. Fuente: Elaboración propia.

`Protegida.jsx` envuelve las rutas de evaluación. Sin sesión, muestra qué es esa evaluación y el botón de Google. Con sesión pero sin términos aceptados, muestra `Bienvenida.jsx`. El backend exige lo mismo con 401; esta pantalla solo evita que el alumno llene un test para descubrirlo al final.

10.2 Estado que guarda el navegador

Tabla 18. Estado guardado en el navegador

La tabla detalla las claves que la interfaz guarda en el almacenamiento del navegador, su contenido y el momento en que se borran.

Clave	Almacenamiento	Contenido	Se borra
`auth`	localStorage	JWT propio y datos de la cuenta	Al cerrar sesión
`holland-perfil`	localStorage	Código, seis áreas y ocupaciones del último Holland	Al cerrar sesión; al iniciar sesión se sincroniza desde `/api/holland/mio`
`holland-borrador`	localStorage	Respuestas a medias del test y página actual	Al calificar o al reiniciar el test
`session-id`	sessionStorage	UUID del recorrido actual	Al cerrar la pestaña o con `nuevaSesion()`
`dev-sin-login`	sessionStorage	Bandera del acceso sin sesión	Solo existe en desarrollo

Tabla 18. Estado guardado en el navegador. Fuente: Elaboración propia.

El `session_id` identifica un recorrido (Holland, chat y tablero), no una carga de página: sobrevive a recargar. "Hacer otro test" y "Responderlo de nuevo" llaman a `nuevaSesion()` para no mezclar dos pruebas.

10.3 El chat (`Chat.jsx`)

- **Fases.** `chat`, `loading` (barra de progreso simulada mientras llega la recomendación) y `dashboard`.
- **Preguntas fijas.** Salen de `preguntas-fijas.js`. Cada una tiene `clave`, `tipo` (`texto` u `opcion`), texto y opciones; texto y opciones pueden ser funciones de lo ya respondido (los grados dependen del nivel). El campo `si` permite saltar una pregunta que no aplica.
- **Validación en la frontera.** Nombre (2 a 40 letras, sin groserías, sin saludo), edad (10 a 99) y carrera cursada (2 a 60). La lista de groserías está duplicada a propósito en `Chat.jsx` y `main.py`.
- **Conteo de adaptativas.** `cuentaAdaptativas` resta las claves fijas, `departamento` y `carrera_descartada`. Contarlas mal cerraba el chat a las 3 preguntas en vez de 4 (defecto detectado en la prueba con alumnos del 2026-08-24).
- **Oferta de resultado.** Al llegar a 4 adaptativas, o antes si la IA responde `terminado`, se ofrece ver la recomendación o seguir hasta 8.

- **Regresar.** Una pila de estados previos (`undoStack`) restaura respuestas, historial y pregunta.
- **Voz.** `POST /api/tts` devuelve MP3; el audio se cachea por texto exacto en un `Map`, y las preguntas fijas se precargan al entrar. Antes de pedir el audio se quitan la negrita y los paréntesis. Si falla, se usa `speechSynthesis`. El tecleo se sintetiza con WebAudio.

10.4 Tablero e informe

`Dashboard.jsx` recibe las carreras y pinta barras (HTML y CSS), el anillo (Recharts), el detalle por carrera y los botones del simulador y el comparador. El mismo componente se reusa desde el historial y desde el registro de administración.

`reporte.js` arma el PDF A4 con jsPDF en el navegador; se carga con `import()` dinámico solo cuando el alumno lo pide.

10.5 Identidad visual

- Paleta de marca: azul, azul marino, gris y negro. Variables en `src/index.css` (`--navy`, `--accent`, `--text`, `--muted`, `--bg`) y 12 tonos para gráficas y PDF en `src/colors.js`.
- Los verdes, rojos y naranjas que quedan son semánticos y no se deben "corregir": el indicador de confianza (semáforo), la escala de caritas de los tests y el nivel de ajuste Best, Great y Good de O*NET.
- Todo texto de la interfaz va en español.

10.6 Compilación

```
cd frontend
npm run lint
npm run build
npm run preview
```

`build` deja el sitio estático en `frontend/dist/`. `preview` lo sirve en el puerto 4173 tal como lo recibirá el alumno, sin las funciones de desarrollo.

11. Integraciones externas

Orienta depende de cuatro servicios externos; si cae uno, el resto de la aplicación sigue funcionando y el usuario recibe un mensaje claro.

Tabla 19. Servicios externos

La tabla resume los cuatro servicios de terceros de los que depende Orienta, cómo se invoca cada uno y qué ocurre cuando deja de responder.

Servicio	Para qué	Cómo se llama	Si falla
Gemini (Google)	Preguntas adaptativas, recomendación, simulador, comparador	SDK `google-genai` desde el backend	Reintentos, key de respaldo y luego 503 con mensaje
O*NET Web Services	Ítems y calificación del test de Holland	`httpx` desde el backend, cabecera `X-API-Key`	502 con "El servicio de O*NET no respondió…"; el chat sigue disponible
Google Identity Services	Inicio de sesión	Botón `@react-oauth/google` en el navegador; verificación con `google-auth` en el backend	Nadie puede iniciar sesión; las páginas informativas siguen abiertas
edge-tts (Microsoft)	Voz neuronal del chat	Biblioteca `edge-tts` desde `POST /api/tts`	El navegador usa su propia voz (`speechSynthesis`)

Tabla 19. Servicios externos. Fuente: Elaboración propia.

11.1 O*NET (test de Holland)

`app/holland.py` es un proxy: el navegador no puede llamar a O*NET directo, porque la key quedaría expuesta y el servicio no envía cabeceras CORS.

- **Host.** `https://api-v2.onetcenter.org/mpp/interestprofiler` (API v2.0). El sitio de documentación, `services.onetcenter.org`, sirve la v1.9 y rechaza la key con 401.
- **Ítems.** `/questions?start=1&end=60`. Sin `start` y `end`, la API devuelve solo 20. El banco se guarda en memoria hasta reiniciar el backend.
- **Calificación.** `/results?answers=<60 dígitos>` para las seis áreas y `/careers?answers=…&zone=4&start=1&end=100` para las ocupaciones que piden preparación universitaria. El parámetro viejo `job_zone` no da error, pero la API lo ignora y devuelve todas las ocupaciones.
- **Código.** Las tres áreas más altas; los empates se resuelven en el orden R, I, A, S, E, C. La letra sale del campo `code` en inglés, no del título traducido.
- **Ocupaciones.** Se toman de los niveles Best, luego Great, luego Good, hasta juntar al menos 10, con tope de 30.
- **Adaptaciones.** Los ítems 14 y 56 se reescriben para Guatemala (cría de animales de granja en vez de criadero de peces; clases de nivel medio en vez de *high school*). El

índice y el área no cambian. Si O*NET cambia el texto original, el backend falla a propósito en lugar de adaptar otra pregunta. Es una desviación del instrumento que debe declararse.

- **Licencia.** O*NET exige acreditar y enlazar el servicio. La página del test lo hace; no quitar ese texto.
- **Uso en el chat.** Si el alumno hizo Holland antes, el chat recibe un bloque con código, puntajes y hasta 8 ocupaciones, y una regla que obliga a nombrar el área más alta en la primera pregunta. Si las dos áreas más altas están a 5 puntos o menos, la primera pregunta debe desempatarlas.

11.2 Google Identity

Un solo cliente de OAuth sirve a los dos lados: el frontend lo usa en `VITE_GOOGLE_CLIENT_ID` y el backend lo usa en `GOOGLE_CLIENT_ID` para verificar el token. En la consola de Google, cada dominio desde el que se sirve el frontend debe estar en los orígenes de JavaScript autorizados.

12. Autenticación, seguridad y privacidad

El alumno entra con Google, el backend emite su propia sesión firmada y cada endpoint que evalúa o muestra datos personales la exige; los secretos viven solo en variables de entorno.

12.1 Flujo de autenticación

1. El botón de Google (Google Identity Services) entrega al navegador un ID token.
2. El navegador lo envía a `POST /api/auth/google`.
3. El backend lo verifica contra `GOOGLE_CLIENT_ID`, busca la cuenta por `google_sub` (estable aunque cambie el correo), la crea si no existe y devuelve un JWT propio: HS256, firmado con `SESSION_SECRET`, con el id del alumno y vigencia de 90 días.
4. El navegador guarda el JWT en `localStorage` y lo envía como `Authorization: Bearer <token>`.
5. La primera vez, `Bienvenida.jsx` pide confirmar el nombre y aceptar los términos; `POST /api/aceptar-terminos` guarda la fecha.

No se usa el ID token de Google como sesión porque vence en una hora.

12.2 Autorización

- **Alumno.** `requiere_login` devuelve la cuenta del JWT. Los endpoints toman el `estudiante_id` de la sesión y nunca del cuerpo de la petición, para que nadie escriba en la fila de otra cuenta.
- **Administrador.** `requiere_admin` compara el correo verificado por Google contra `ADMIN_EMAILS`. Sin la variable no entra nadie. El menú pregunta a `/api/admin/soy` solo para decidir si muestra el enlace; quien decide el acceso es el backend.
- **Modo local.** `LOGIN_OPCIONAL=1` deja evaluarse sin sesión como `dev@local`. Solo el valor `1` exacto lo activa, y ese usuario nunca es administrador. En producción la variable no se define.

12.3 Defensas implementadas

- **CORS cerrado.** Solo los orígenes de `ORIGENES_PERMITIDOS`; nunca `*`.
- **Validación en la frontera.** Pydantic valida forma y tamaño de toda entrada: nombre (2 a 40 caracteres, sin groserías), `session_id` (máximo 64), cadena de Holland (60 dígitos del 1 al 5), perfil de Holland que llega del navegador (código, seis áreas de 0 a 40, hasta 12 ocupaciones de 120 caracteres), juicio (lista cerrada) y nota (4,000 caracteres).
- **Inyección de prompt.** Todo prompt termina con un bloque que declara que lo escrito por el alumno son datos, nunca instrucciones. El texto que llega del navegador (por ejemplo, el perfil de Holland) lo vuelve a redactar el backend antes de meterlo al prompt.
- **Salida de la IA.** Se valida contra un esquema; el frontend dibuja negrita y cursiva partiendo el texto, sin inyectar HTML.
- **Secretos.** `backend/.env` está en `.gitignore`. La key de O*NET y la de Gemini nunca llegan al navegador: O*NET pasa por el proxy del backend.
- **Límites de gasto.** Tope de chats por alumno y tope diario global de tokens (`cuota.py`).

12.4 Datos personales

Tabla 20. Datos personales y quién los ve

La tabla identifica los datos personales que maneja el sistema, el lugar donde se almacenan y las personas o servicios que tienen acceso a ellos.

Dato	Dónde vive	Quién lo ve
Nombre y correo de Google	`estudiantes`	El alumno, los administradores
Respuestas del chat (incluye edad, grado y carrera cursada)	`respuestas_cuestionario`	El alumno, los administradores
Recomendación y juicio del profesional	`respuestas_cuestionario`	El alumno ve su recomendación; el juicio solo los administradores
Resultados de Holland	`resultados_holland`	El alumno, los administradores
Respuestas enviadas a Gemini	Servidores de Google	Google, según sus términos de la API
Respuestas de Holland (60 dígitos)	Servidores de O*NET	O*NET; no incluyen datos personales

Tabla 20. Datos personales y quién los ve. Fuente: Elaboración propia.

La mayoría de los alumnos son menores de edad. Los términos de uso prometen reserva, conservación mientras dure la investigación y borrado a pedido (procedimiento en la sección 7.6).

12.5 Riesgos conocidos y mejoras recomendadas

Tabla 21. Riesgos conocidos y mejoras recomendadas

La tabla registra los riesgos de seguridad y privacidad detectados en la versión actual, con el detalle de cada uno y la mejora que se sugiere aplicar.

Riesgo	Detalle	Mejora sugerida
El nombre viaja a Gemini	`preguntas._historial` agrega "El estudiante se llama …" y `recomendar` y `extras` mandan todas las respuestas, incluido `nombre`. Contradice el formato de consentimiento, que declara que el envío no incluye el nombre	Quitar la clave `nombre` del perfil antes de armar cada prompt y repetir la medición de calidad con brazo de control
`GET /api/uso-tokens` es público	Expone los `session_id` y el consumo por sesión	Protegerlo con `requiere_admin`
`POST /api/tts` es público	Cualquiera puede usarlo como servicio de voz gratuito	Exigir sesión o limitar el largo del texto
JWT en `localStorage`	Un XSS podría leerlo. Hoy no hay HTML inyectado, pero es la superficie	Cookie `HttpOnly` si el sistema crece
Estado en memoria	Cobertura de dimensiones, asignación de keys y caché viven en el proceso	Moverlos a la base de datos si hay más de una instancia
Lista de groserías	Curada, no exhaustiva; no detecta evasiones	Suficiente para el nombre; no es

Riesgo	Detalle	Mejora sugerida
		moderación completa

Tabla 21. Riesgos conocidos y mejoras recomendadas. Fuente: Elaboración propia.

13. Despliegue en producción y monitoreo

Producción corre desde el 2026-08-20 en tres piezas: el sitio estático y la API en Render, y la base de datos en Neon; cada confirmación en la rama principal del repositorio MiOrienta se despliega sola.

13.1 Piezas

Tabla 22. Piezas del entorno de producción

La tabla identifica los tres componentes desplegados de Orienta, el servicio que aloja a cada uno, su dirección y el plan contratado.

Pieza	Servicio	Dirección	Plan
Sitio (frontend)	Render, sitio estático	https://miorienta.com (dominio en Namecheap)	Gratuito
API (backend)	Render, servicio web Python	https://miorienta.onrender.com	Gratuito
Base de datos	Neon, PostgreSQL administrado, región us-west-2	Conexión directa, sin pooler	Gratuito

Tabla 22. Piezas del entorno de producción. Fuente: Elaboración propia.

La base no está en Render porque la de su plan gratuito se borra a los 30 días. Neon no vence y suspende el cómputo tras 5 minutos sin uso; despierta solo en la siguiente conexión.

13.2 Montaje desde cero

Se crean en este orden, porque cada pieza necesita la URL de la anterior. El repositorio de producción trae `render.yaml` como referencia, pero los Blueprints de Render piden plan de pago: en el plan gratuito se crean a mano.

1. **Base de datos.** Cree un proyecto en Neon en la misma región que la API. En **Connect**, apague **Connection pooling** antes de copiar la cadena: el pooler trabaja por transacción y no lleva bien las sentencias preparadas de psycopg. Copie la cadena completa, con `?sslmode=require`.
2. **API.** En Render, **New > Web Service** con el repositorio de producción:

Tabla 23. Configuración del servicio web de la API

La tabla indica los valores con que se crea en Render el servicio que ejecuta el backend.

Campo	Valor
Root Directory	`backend`
Language	Python 3
Build Command	`pip install -r requirements.txt`
Start Command	`python seed_carreras.py && uvicorn app.main:app --host 0.0.0.0 --port $PORT`
Health Check Path	`/health`

Tabla 23. Configuración del servicio web de la API. Fuente: Elaboración propia.

Defina `DATABASE_URL`, `GEMINI_API_KEY`, `ONET_API_KEY`, `GOOGLE_CLIENT_ID`, `SESSION_SECRET`, `ADMIN_EMAILS`, `MAX_CHATS_DIARIOS` y `TOPE_TOKENS_DIARIO`. No defina `LOGIN_OPCIONAL`. Al terminar, `GET /health` debe responder `{"status":"ok"}`.

3. **Sitio. New > Static Site** con el mismo repositorio: Root Directory `frontend`, Build Command `npm ci && npm run build`, Publish Directory `dist`. Variables: `VITE_API_URL` (la URL de la API) y `VITE_GOOGLE_CLIENT_ID`. En **Redirects/Rewrites**, una regla de origen `/*`, destino `/index.html`, acción **Rewrite**; sin ella, entrar directo a `/holland` o recargar da 404.
4. **Cabos finales.** Ponga la URL del sitio en `ORIGENES_PERMITIDOS` de la API, y agréguela a los orígenes de JavaScript autorizados del cliente OAuth de Google. Faltando lo primero, la app carga pero no responde (síntoma típico: catálogo vacío). Faltando lo segundo, el botón de Google no aparece.

El catálogo se carga en cada arranque de la API: `seed_carreras.py` es idempotente y el plan gratuito no da consola.

13.3 Publicar un cambio

1. Haga y pruebe el cambio en el repositorio de trabajo (AsistenteVocacional).
2. Aplíquelo también en el repositorio de producción (MiOrienta): las historias son independientes.
3. Si cambiaron dependencias de Python, regenere el archivo que instala Render:

```
cd backend
uv export --no-hashes --no-emit-project --format requirements-txt -o
requirements.txt
```

4. Confirme y suba a la rama principal de MiOrienta. Render compila el sitio y reinicia la API.
5. Verifique `/health`, inicie sesión y haga un recorrido completo (Holland, chat y PDF).

Si cambia `VITE_API_URL` o `VITE_GOOGLE_CLIENT_ID`, hay que volver a desplegar el sitio: guardarlas en el panel no basta. Guarde los `.env` en UTF-8 sin BOM; con BOM, la primera variable no se lee y el backend cae a su valor por omisión sin avisar.

13.4 Mantener la API despierta

En el plan gratuito, Render duerme la API tras 15 minutos sin tráfico y la primera visita espera cerca de un minuto. Un trabajo programado en cron-job.org pide `/health` cada 10 minutos (`*/10 0-1,3-23 * * *`, hora de Guatemala). Se usa `/health` a propósito: no toca la base, así Neon sigue suspendida y no gasta.

cron-job.org marca cada ejecución en rojo ("salida demasiado grande") porque Render responde sin `Content-Length`. Se ignora: la prueba real son las líneas `GET /health HTTP/1.1 200 OK` en los registros de Render.

13.5 Monitoreo

Tabla 24. Puntos de monitoreo

La tabla señala qué aspectos del sistema conviene vigilar en producción y dónde se consulta la información de cada uno.

Qué vigilar	Dónde
Errores y reintentos de Gemini	Registros de Render, líneas `[gemini]`
Dimensión que cubre cada pregunta y reintentos de cierre	Registros, líneas `[dimension]`
Recorte del catálogo	Registros, líneas `[filtro]`
Fallas de voz, O*NET o filtros de seguridad	Registros, líneas `[tts]`, `[holland]`, `[seguridad]`
Consumo de tokens	`GET /api/uso-tokens`, o `uv run python gasto_24h.py` con la `DATABASE_URL` de producción
Gasto real, incluido el almacenamiento de caché	Facturación de Google Cloud
Disponibilidad	Historial de cron-job.org o UptimeRobot; probar con `curl -I https://miorienta.onrender.com/health`

Tabla 24. Puntos de monitoreo. Fuente: Elaboración propia.

13.6 Límites en producción

- `MAX_CHATS_DIARIOS=3`: chats terminados por alumno en una ventana deslizante de 24 horas.
- `TOPE_TOKENS_DIARIO=6000000` desde el 2026-08-25. Una sesión completa gasta unos 65,000 tokens, así que alcanza para unas 90 evaluaciones por día. Este tope se cuenta por día UTC (desde la medianoche UTC, las 18:00 de Guatemala). Si se agota, el alumno recibe un 429 a media conversación: antes de una sesión con un salón entero, revise y suba el tope.
- Holland no tiene límite y no pasa por el presupuesto global.

14. Pruebas y metodología de medición

La lógica determinista se prueba con autocomprobaciones que no usan red ni gastan cuota; el comportamiento del modelo de lenguaje, que no es determinista, se mide con experimentos A/B con brazo de control.

14.1 Autocomprobaciones del backend

Cada módulo trae un bloque `if __name__ == "__main__"` con aserciones. Se ejecutan desde `backend/`:

```
uv run python -m app.auth
MAX_CHATS_DIARIOS=3 uv run python -m app.cuota
uv run python -m app.db
uv run python -m app.filtro
uv run python -m app.preguntas
uv run python -m app.recomendar
uv run python -m app.diversificado
uv run python -m app.holland_filtro
uv run python -m app.main
```

Tabla 25. Autocomprobaciones del backend

La tabla describe qué verifica la autocomprobación de cada módulo, todas ejecutables sin red y sin consumir cuota del motor de IA.

Módulo	Qué verifica
`auth`	Emisión y verificación del JWT, firma ajena o vencida, lista de administradores, que el modo local solo se active con `1`
`cuota`	Cálculo de la espera del límite de chats y el texto que ve el alumno
`db`	Normalización de la URL de la base al controlador psycopg 3
`filtro`	Pre-filtro y descarte de la carrera abandonada
`preguntas`	Historial, cobertura por sesión, conteo de adaptativas y la guardia de cierre con un doble de `generar()`
`recomendar`	Texto del catálogo, agrupado de sedes y paso a la key de respaldo ante un 429 simulado
`diversificado`	Sugerencias para básicos
`holland_filtro`	Que las 90 carreras tengan vector RIASEC y el flag esté apagado
`main`	Cruce del Holland de cada evaluación con el registro

Tabla 25. Autocomprobaciones del backend. Fuente: Elaboración propia.

`app.cuota` toma el límite de `backend/.env`. Con `MAX_CHATS_DIARIOS=0`, el valor recomendado para desarrollo, su autocomprobación falla; por eso el comando lo fija en 3. Los módulos de desarrollo (`psicometrico`, `personalidad`, `cip_fogliatto`, `cip`, `cip_filtro`) tienen sus propias autocomprobaciones. `app.holland` hace llamadas reales a O*NET si hay key.

14.2 Autocomprobaciones del frontend

```
cd frontend
node src/preguntas-fijas.js
node check-ritmo.mjs
npm run lint
```

`preguntas-fijas.js` verifica que se quite el saludo del nombre ("Hola soy Yesi" queda "Yesi", pero "Holanda" no se toca). `check-ritmo.mjs` extrae dos funciones de `Chat.jsx` y verifica el tiempo de lectura sin voz y que silenciar no cuelgue el turno.

14.3 Pruebas de integración

No están automatizadas: requerirían un entorno con sus propias credenciales de los cuatro servicios externos. Antes de cada publicación se hace a mano un recorrido completo contra el entorno real:

1. Iniciar sesión con una cuenta nueva y aceptar los términos.
2. Hacer el test de Holland y seguir al chat.

3. Responder las fijas y al menos cuatro adaptativas; confirmar que aparece la etiqueta de Holland.
4. Ver el tablero, abrir el simulador y el comparador, descargar el PDF.
5. Revisar el historial y, con una cuenta de administrador, el registro.
6. Marcar esa evaluación como **Fuera del estudio**.

14.4 Medición de cambios en el motor

Los scripts `experimento_*.py` de `backend/` corren perfiles simulados contra Gemini, guardan los resultados crudos en `data/tests/` y su análisis va en `experiments/`. **Gastan cuota.**

Reglas, aprendidas a costa de resultados engañosos:

- **Piso de ruido.** Con entrada idéntica, correr la misma configuración dos veces cambia el top-1 en 2 a 3 de cada 4 casos. Toda medición lleva un brazo de control (la configuración actual corrida otra vez) y solo cuenta un efecto mayor que ese ruido.
- **Perfiles inestables.** Hay perfiles que son "una moneda al aire" (por ejemplo, empates técnicos entre dos áreas RIASEC opuestas); un A/B que los use necesita muchas repeticiones.
- **Si mide peor, se revierte** y se documenta en `experiments/` con las transcripciones.
- Los análisis que no llaman a la API (`analiza_filtro.py`, `cobertura_banco.py`, `costo_por_departamento.py`, `dump_prompt.py`) se pueden repetir sin costo.

14.5 Validación con alumnos reales

El criterio externo es el juicio de la psicóloga en el registro de administración (`acerto`, `parcial`, `no_acerto`). La primera corrida en campo fue el 2026-08-24 con 18 alumnos y dejó dos defectos ya corregidos: el conteo de adaptativas y la cobertura que no se limpiaba entre pruebas. El plan del estudio está en `docs/estudio-con-estudiantes.md`.

15. Mantenimiento

El catálogo es la fuente de verdad: agregar carreras o centros es editar archivos JSON y volver a cargar, sin tocar código ni prompts.

15.1 Estado actual del catálogo

147 carreras distintas en 202 programas por sede, de 10 universidades con 12 centros: 9 en Quetzaltenango (185 programas) y 3 en Totonicapán (17). Todas las universidades con sede física en esos dos departamentos ya están cargadas. La IA ve 90 perfiles distintos, porque las sedes de una misma carrera comparten perfil.

15.2 Formato de un archivo de centro

Un archivo `backend/data/carreras_<centro>.json` por centro universitario:

```
{
  "departamento": "Totonicapán",
  "centro": "CUNTOTO",
  "universidad": "USAC",
  "carreras": [
    {
      "nombre": "Nombre oficial de la carrera",
      "perfil": "Arquetipo: El …. AFINIDAD (ser): …. HABILIDADES (saber hacer): …. ENTORNO: …. GUSTOS TEMÁTICOS: …. ESTILO COGNITIVO: …."
    },
    {
      "nombre": "Profesorado en Ciencias Sociales y Formación Ciudadana",
      "perfil_id": "pedagogia_ciencias_sociales",
      "sello": "Lo que distingue a esta sede, en una o dos frases."
    }
  ]
}
```

- **`perfil`** (texto) es lo único que la IA lee de la carrera. Debe empezar con `Arquetipo: <nombre>.`: de ahí salen la frase del catálogo público y el enfoque por omisión de la sede.
- **`perfil_id`** sustituye a `perfil` cuando la misma carrera existe en varias sedes. Apunta a una clave de `data/perfiles_compartidos.json`; el perfil viaja una sola vez en el prompt.
- **`sello`** es opcional y se muestra en "¿Dónde estudiarla?". La IA no lo recibe.

15.3 Agregar un centro en un departamento existente

1. Cree el archivo JSON del centro. Reuse `perfil_id` existentes para las carreras que ya están en otras sedes.
2. Cargue: `cd backend` y `uv run python seed_carreras.py`. Imprime cuántas carreras son nuevas y cuántas se actualizaron. En producción se carga solo al arrancar la API.

3. Si el backend imprime `AVISO: '<nombre>' choca con otra carrera de nombre idéntico`, dos perfiles distintos tienen el mismo nombre visible y uno quedaría oculto para la IA: corrija el `perfil_id`.
4. Para carreras con perfil nuevo, agregue su código RIASEC a `data/holland_catalogo.json` con `codificar_holland.py`; si no, no aparecerán entre las "afines a tu perfil" de Holland.
5. Revise `data/diversificados.json` si alguna carrera nueva necesita una clave para conectar con un diversificado.

15.4 Agregar un departamento nuevo

Además de los pasos anteriores:

1. Agregue el nombre del departamento al conjunto `ACTIVOS` de `frontend/src/Mapa.jsx`, escrito igual que en `src/data/guatemalaDeptos.js` y en el JSON. El encuadre del mapa se recalcula solo.
2. Investigue la oferta real de diversificado del departamento y actualice `departamentos` en `data/diversificados.json`: la tabla actual solo cubre Quetzaltenango y Totonicapán.
3. Tenga en cuenta el costo: el catálogo es cerca del 97% de los tokens de cada llamada, así que cada centro nuevo encarece la recomendación en proporción. `uv run python costo_por_departamento.py` lo estima sin gastar cuota.

15.5 Renombrar o quitar una carrera

`seed_carreras.py` actualiza por `(centro, nombre)` y **nunca borra**. Si se renombra una carrera en el JSON, la fila vieja queda en la base y la IA vería las dos. Después de renombrar o quitar, borre la fila sobrante:

```
DELETE FROM carreras WHERE centro = '<centro>' AND nombre = '<nombre viejo>';
```

Las recomendaciones ya guardadas no se rompen: guardan el nombre, no el id.

15.6 Preguntas fijas y banco de temas

El texto y las opciones de las preguntas fijas están en `frontend/src/preguntas-fijas.js`. Las etiquetas se redactan para que el alumno se reconozca, no para alimentar al pre-filtro, y nunca nombran carreras. El banco de temas tiene 25 opciones en 6 grupos con tope de 5; ese cambio

está aplicado pero sin medir (`experiments/banco-de-opciones.md`). Cualquier cambio a estas etiquetas cambia la señal que recibe la IA y debe medirse.

15.7 Dependencias y modelos

- **Python.** `uv add <paquete>` o `uv lock --upgrade`, y luego regenerar `requirements.txt` en el repositorio de producción (sección 13.3).
- **Node.** `npm install <paquete>` o `npm update` en `frontend/`; Render usa `npm ci` con el `package-lock.json`.
- **Modelo de Gemini.** Antes de cambiar `GEMINI_MODEL` se mide: `gemini-3.5-flash-lite` cambió 2 de 5 top-1, leyó peor la señal indirecta y costó entre 25% y 40% más (`experiments/comparacion-modelos.md`).
- **Ítems de O*NET.** Si O*NET cambia la redacción de los ítems 14 o 56, `/api/holland/preguntas` falla a propósito; actualice `ADAPTACIONES` en `app/holland.py`.

16. Costos y cuotas

El único servicio que cobra por uso es Gemini; una evaluación completa cuesta centavos de dólar, y el caso más caro medido fue el caché explícito con tráfico disperso.

16.1 Precios de Gemini usados en los cálculos

Modelo `gemini-3.1-flash-lite`, en dólares por millón de tokens, confirmados contra la factura de agosto de 2026:

Tabla 26. Precios de Gemini usados en los cálculos

La tabla muestra el precio por millón de tokens de cada concepto que factura Gemini, confirmado contra la factura de agosto de 2026.

Concepto	Precio
Entrada	$0.25
Salida	$1.50
Entrada leída del caché	$0.025
Almacenamiento del caché explícito	$1.00 por millón de tokens por hora

Tabla 26. Precios de Gemini usados en los cálculos. Fuente: Elaboración propia con datos de la facturación de Google Cloud.

Los tres primeros están en `PRECIO_USD_POR_1M` de `recomendar.py`, que usan los reportes de gasto.

16.2 Costo por alumno

Una sesión completa (cuatro o más adaptativas y la recomendación) gasta unos 65,000 tokens.

Tabla 27. Costo por alumno según el escenario

La tabla compara el costo medido de una evaluación completa con y sin caché, y según el departamento y la concentración del tráfico.

Escenario	Costo por alumno	Fuente
Sin ningún caché (sesión de 55,837 tokens, 2026-08-11)	$0.017	`decisions/gemini-costos-y-caching.md`
Caché explícito, una clase entera a la misma hora	$0.006	`decisions/gemini-costos-y-caching.md`
Caché explícito, alumnos dispersos, Totonicapán	$0.014	`experiments/cache-compartido.md`
Caché explícito, alumnos dispersos, Quetzaltenango	$0.044	`experiments/cache-compartido.md`

Tabla 27. Costo por alumno según el escenario. Fuente: Elaboración propia.

El caché explícito solo ahorra cuando muchos alumnos usan la app en la misma hora; con tráfico disperso el alquiler supera el ahorro. Por eso está apagado por omisión (`CACHE_EXPLICITO=0`).

En la factura del 19 al 31 de agosto de 2026 ($1.59 en total), el almacenamiento del caché fue $0.60, el 38%, y la línea más cara. Ese costo no aparece en `uso_tokens`: el gasto real solo se ve en la facturación de Google Cloud.

16.3 Cuotas

- **Gemini, nivel gratuito.** Alrededor de 15 solicitudes por minuto y 500 por día, por proyecto. Con un salón entero, los alumnos esperan los reintentos; para una sesión de clase conviene una key con facturación como principal.

16.4 Otros servicios

Tabla 28. Costo y límites de otros servicios

La tabla resume el plan, el costo y el límite relevante de los servicios complementarios que sostienen la operación de Orienta.

Servicio	Plan	Costo	Límite que importa
O*NET Web Services	Licencia de desarrollador	Gratis	Exige acreditar el servicio en pantalla
Render (sitio y API)	Gratuito	Gratis	750 horas al mes; la API se duerme sin tráfico
Neon (PostgreSQL)	Gratuito	Gratis	Suspende el cómputo sin uso; horas de cómputo limitadas
edge-tts	API no oficial	Gratis	Puede dejar de funcionar sin aviso
Dominio miorienta.com	Namecheap	Renovación anual	Vence si no se renueva

Tabla 28. Costo y límites de otros servicios. Fuente: Elaboración propia.

17. Solución de problemas técnicos

La mayoría de las fallas vistas en este proyecto fueron de configuración (orígenes, variables congeladas en el build, keys), no de código.

Tabla 29. Solución de problemas técnicos

La tabla relaciona cada síntoma observado durante el desarrollo y la operación con su causa probable y la solución que lo resuelve.

Síntoma	Causa probable	Solución
La app carga pero el catálogo sale vacío y nada responde	El origen del sitio no está en `ORIGENES_PERMITIDOS`; el navegador bloquea por CORS	Agregar la URL exacta del sitio y reiniciar la API
El botón de Google no aparece o falla	Falta `VITE_GOOGLE_CLIENT_ID`, o el origen no está autorizado en el cliente OAuth	Configurar ambos y volver a compilar el sitio
`/api/auth/google` responde 503	Falta `GOOGLE_CLIENT_ID` o `SESSION_SECRET`	Definirlos en el backend
Endpoints de IA responden 503 "Falta configurar GEMINI_API_KEY"	La variable no está o el `.env` tiene BOM	Definirla; guardar el `.env` en UTF-8 sin BOM
Gemini responde 404 sin cambios en el código	Google retiró el modelo	Actualizar `GEMINI_MODEL` y `GEMINI_MODEL_FINAL`, previa medición
Muchos 429 o esperas largas con un salón entero	Cuota por minuto del nivel gratuito	Key con facturación como principal; revisar `TOPE_TOKENS_DIARIO`
"El asistente alcanzó su límite de uso por hoy"	Se llegó a `TOPE_TOKENS_DIARIO`	Subir el tope o esperar a la medianoche UTC

Síntoma	Causa probable	Solución
Holland responde 503	Falta `ONET_API_KEY`	Definirla
Holland responde 401 desde O*NET	Se apunta al host de la v1.9	El host correcto es `api-v2.onetcenter.org`
`/api/holland/preguntas` falla con "Revisar app.holland.ADAPTACIONES"	O*NET cambió el texto de un ítem adaptado	Actualizar `ADAPTACIONES`
Entrar directo a `/holland` o recargar da 404 en producción	Falta la regla de reescritura del sitio	Regla `/*` a `/index.html`, acción Rewrite
La primera visita tarda cerca de un minuto	La API de Render estaba dormida	Mantenerla despierta (sección 13.4)
Errores de sentencias preparadas con Neon	Conexión por el pooler	Usar la conexión directa
Render instala dependencias viejas	`requirements.txt` desactualizado	Regenerarlo con `uv export`
Una columna nueva no aparece	`create_all` no altera tablas existentes	Agregar su `ALTER TABLE … ADD COLUMN IF NOT EXISTS` en `lifespan()`
No suena la voz	edge-tts falló o el navegador bloqueó el audio	El navegador usa su voz; revisar líneas `[tts]` en los registros
`start.ps1` no conecta con Docker	Docker Desktop no arrancó en 2 minutos	Abrirlo a mano y repetir
El puerto 5432, 8000 o 5173 está ocupado	Otro proceso los usa	`.\stop.ps1`, o cerrar el proceso que los ocupa

Tabla 29. Solución de problemas técnicos. Fuente: Elaboración propia.

18. Decisiones técnicas y limitaciones conocidas

Cada decisión de peso tiene su registro en `decisions/` o `experiments/` con el motivo y la evidencia; antes de revertir una, hay que leerlo.

18.1 Decisiones principales

Tabla 30. Decisiones técnicas principales

La tabla reúne las decisiones de diseño de mayor peso, el motivo de cada una y el documento del repositorio donde se registró la evidencia.

Decisión	Motivo	Referencia
Recomendación con un modelo de lenguaje, no con una red entrenada	No hay datos etiquetados; entrenar con datos simulados sería circular; el modelo entiende texto libre	`decisions/llm-vs-red-neuronal.md`
Prompts agnósticos del catálogo	Agregar carreras no exige tocar código	`docs/catalogo.md`
Preguntas fijas sin IA más 4 a 8 adaptativas	Menos llamadas y menos costo; quitar las fijas midió peor	`experiments/holland-en-chat.md`, `experiments/psicometrico-en-chat.md`
Vector de cobertura de dimensiones con guardia en código	Cumplimiento del mínimo de preguntas de 40% a 100%	`experiments/cobertura-dimensiones.md`

Decisión	Motivo	Referencia
Preguntas por microexperiencias, revertidas	Midieron 6/10 contra 10/10	`experiments/microexperiencias.md`
Pre-filtro sin IA solo en las adaptativas	Reduce tokens; no cambia la recomendación más allá del ruido	`decisions/filtro-catalogo.md`, `experiments/filtro-catalogo-ab.md`
Instituciones adjuntadas por Python, no por la IA	Ahorra tokens y evita que la IA reescriba datos reales	`decisions/gemini-costos-y-caching.md`
Holland servido por O*NET, sin reimplementarlo	Instrumento avalado y calificación oficial	`docs/holland.md`
Holland no alimenta la recomendación	Medido tres veces: el ranking no se movió	`experiments/holland-en-chat.md`, `experiments/holland-estructura.md`
CIP fuera del menú (2026-08-16)	Sin autorización escrita de uso	`docs/holland.md`
Psicométrico y perfil corto solo en desarrollo	No está medido si mejoran la recomendación	`docs/psicometrico.md`, `docs/personalidad.md`
Inicio de sesión obligatorio y dos topes de uso (2026-08-20)	Proteger el crédito de Gemini y sostener el historial	`docs/api.md`
Caché explícito opcional y apagado (2026-08-24)	El alquiler fue el 38% de la factura	`experiments/cache-compartido.md`
Se mantiene `gemini-3.1-flash-lite` (2026-08-17)	La alternativa cambió resultados y costó más	`experiments/comparacion-modelos.md`

Tabla 30. Decisiones técnicas principales. Fuente: Elaboración propia.

Los comentarios `ponytail:` del código marcan simplificaciones deliberadas con su techo conocido (por ejemplo, el estado en memoria o `create_all` sin migraciones). Lea el comentario antes de "arreglarlas".

18.2 Limitaciones conocidas

- **Alcance geográfico.** Solo Quetzaltenango y Totonicapán.
- **No determinismo.** Con entrada idéntica, el top-1 cambia en 2 a 3 de cada 4 corridas. El resultado se presenta como orientación, no como veredicto.
- **Dependencia externa.** La disponibilidad, los precios y los modelos de Gemini los decide Google; edge-tts es una API no oficial.
- **Pruebas.** No hay pruebas automáticas de integración ni de interfaz.
- **Validación.** El juicio profesional cubre solo una parte de las evaluaciones del estudio piloto.
- **Holland.** Dos ítems se reescribieron para Guatemala; es una desviación del instrumento estandarizado y las ocupaciones que devuelve son del mercado de EE. UU.
- **Banco de temas.** La versión de 25 opciones con tope de 5 está aplicada sin medir.

- **Privacidad.** El nombre que escribe el alumno viaja a Gemini (sección 12.5); corregirlo es el pendiente de mayor prioridad.